

React Application Deployment on AWS EC2 Using Docker

Step 1 — Open PowerShell

Open **PowerShell** from Windows.

You will see:

```
PS C:\WINDOWS\System32>
```

Step 2 — Go to the Downloads Folder

The EC2 **.pem** key is stored in Downloads.

```
cd "C:\Users\DELL\Downloads"
```

Check the files:

```
dir
```

You should see:

```
react-deployment.pem
```

Important: Never share the contents of the **.pem** file.

Step 3 — Connect to AWS EC2

Our EC2 public IP is:

```
52.66.238.68
```

Connect using:

```
ssh -i "C:\Users\DELL\Downloads\react-deployment.pem" ubuntu@52.66.238.68
```

If successful:

ubuntu@ip-172-31-15-190:~\$

You are now inside the Ubuntu EC2 server.

Step 4 — Check Docker

Run:

```
docker --version
```

Check containers:

```
docker ps
```

Check images:

```
docker images
```

Step 5 — Install Docker on EC2

If Docker is not installed:

```
sudo apt update
```

```
sudo apt install -y docker.io
```

Check:

```
docker --version
```

Start Docker:

```
sudo systemctl enable --now docker
```

Check the service:

```
sudo systemctl status docker
```

Look for:

Active: active (running)

Press:

q

to exit the status screen.

Step 6 — Allow Ubuntu User to Run Docker

Run:

```
sudo usermod -aG docker $USER
```

Exit EC2:

```
exit
```

Reconnect from PowerShell:

```
ssh -i "C:\Users\DELL\Downloads\react-deployment.pem" ubuntu@52.66.238.68
```

Test:

```
docker ps
```

If there is no permission error, Docker is ready.

Step 7 — Open the React Project in VS Code

Now we temporarily work on the **Windows machine**, not inside EC2.

Open your React project in VS Code.

Example:

```
D:\CI_In_Git\ci_Demo
```

Open the VS Code terminal.

Check that you are inside the project:

pwd

You should see something similar to:

D:\CI_In_Git\ci_Demo

Step 8 — Login to Docker Hub

From the VS Code terminal:

docker login

Enter:

Username: rakshith006

For the password, use your **Docker Hub Access Token**.

If successful:

Login Succeeded

Step 9 — Build the React Docker Image

Make sure you are inside the React project folder containing the **Dockerfile**.

Run:

docker build -t rakshith006/reactapp:latest .

This will:

1. Read the **Dockerfile**
2. Install the React dependencies
3. Build the React application
4. Create the Docker image
5. Tag it as:

rakshith006/reactapp:latest

Step 10 — Verify the Docker Image

Run:

```
docker images
```

You should see:

REPOSITORY	TAG	IMAGE ID
rakshith006/reactapp	latest	...

Step 11 — Push the Image to Docker Hub

Now push the image:

```
docker push rakshith006/reactapp:latest
```

Wait for all layers to upload.

A successful push will end with something similar to:

```
latest: digest: sha256:...
```

Now your image is available on Docker Hub:

```
rakshith006/reactapp:latest
```

Important

If you get an error such as:

```
EOF
```

during the push, simply retry:

```
docker push rakshith006/reactapp:latest
```

Docker will reuse layers that were already uploaded.

Step 12 — Return to the EC2 Terminal

Go back to your EC2 SSH terminal.

You should see:

```
ubuntu@ip-172-31-15-190:~$
```

Now pull the image from Docker Hub:

```
docker pull rakshith006/reactapp:latest
```

You should eventually see:

```
Status: Downloaded newer image for rakshith006/reactapp:latest
```

Step 13 — Verify the Image on EC2

Run:

```
docker images
```

You should see:

REPOSITORY	TAG	IMAGE ID
rakshith006/reactapp	latest	...

Step 14 — Run the React Container

Run:

```
docker run -d --name reactapp -p 80:80 rakshith006/reactapp:latest
```

What does `-p 80:80` mean?

EC2 Port 80



Docker Container Port 80



Nginx



React Application

Step 15 — Check the Container

Run:

```
docker ps
```

You should see:

CONTAINER ID	IMAGE	STATUS	PORTS
xxxxxxxxxxxx	rakshith006/reactapp:latest	Up ...	0.0.0.0:80->80/tcp

Step 16 — Test Inside EC2

Run:

```
curl http://localhost
```

You should receive HTML content from the React application.

This confirms that the application is running inside the EC2 Docker container.

Step 17 — Configure AWS Security Group

Go to:

AWS Console → **EC2** → **Instances** → **Select Instance** → **Security** → **Security Groups**

Under **Inbound rules**, make sure you have:

Type	Port	Source
SSH	22	Your IP
HTTP	80	0.0.0.0/ 0

If HTTP is missing, add:

Type: HTTP

Port: 80
Source: Anywhere-IPv4

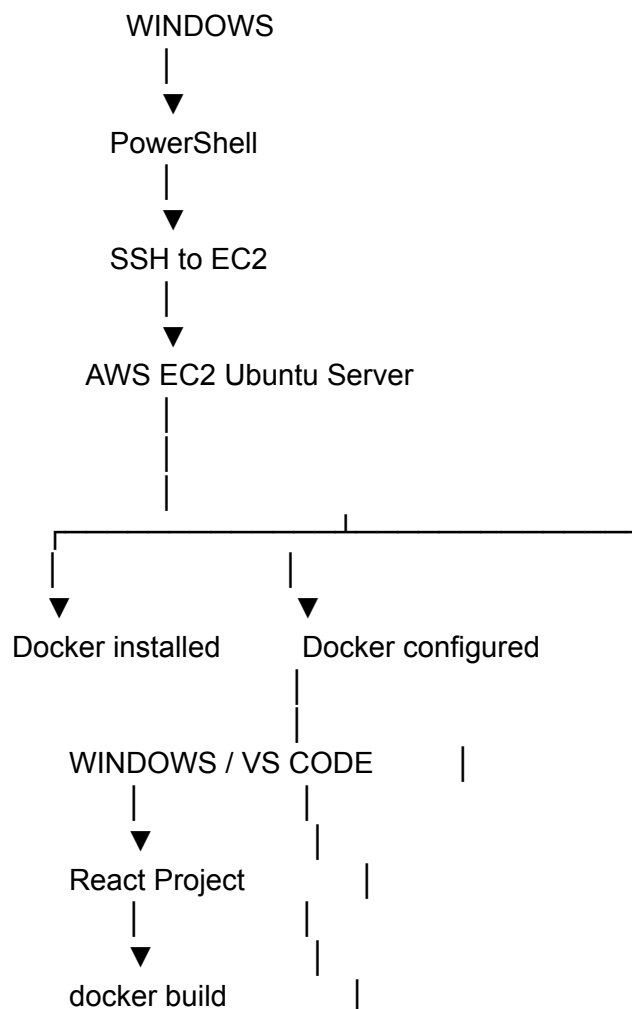
Step 18 — Open the Application

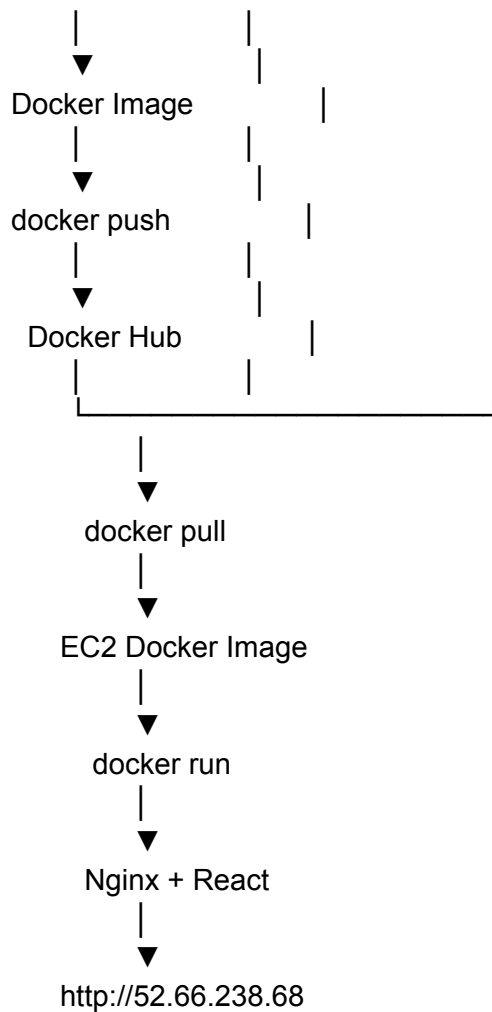
Open your browser on Windows:

`http://52.66.238.68`

Your React application should now appear.

Complete Deployment Flow





Commands in order

Windows → PowerShell:

```
cd "C:\Users\DELL\Downloads"  
ssh -i "C:\Users\DELL\Downloads\react-deployment.pem" ubuntu@52.66.238.68
```

EC2:

```
docker --version  
docker ps
```

Windows → VS Code terminal:

```
docker login  
docker build -t rakshith006/reactapp:latest .  
docker images  
docker push rakshith006/reactapp:latest
```

Back to EC2:

```
docker pull rakshith006/reactapp:latest
docker images
docker run -d --name reactapp -p 80:80 rakshith006/reactapp:latest
docker ps
curl http://localhost
```

Browser:

<http://52.66.238.68>

This gives you a clean **manual Docker deployment workflow** to teach before moving into the automated **GitHub Actions CI/CD pipeline**.

Now that the **manual deployment works**, the next step is to automate it with **GitHub Actions CI/CD**.

Your target flow will be:

```
Developer
↓
GitHub Push
↓
GitHub Actions
↓
Build React
↓
Build Docker Image
↓
Push Image to Docker Hub
↓
SSH into EC2
↓
Pull New Image
↓
Stop Old Container
↓
Start New Container
↓
Updated React App
```

Step 1 — Update the EC2 Host Secret

Because you created a **new EC2 instance**, the public IP changed.

Go to:

GitHub → Your Repository → Settings → Secrets and variables → Actions

Find:

EC2_HOST

Update its value to:

52.66.238.68

Keep:

EC2_USERNAME = ubuntu

Your **EC2_SSH_KEY** should remain the private key corresponding to this EC2 instance. Since you successfully connected using **react-deployment.pem**, make sure the GitHub secret contains the contents of that same key.

Step 2 — Check Your GitHub Secrets

You should have these four secrets:

Secret	Value
DOCKERHUB_USER NAME	rakshith006
DOCKERHUB_Token	Docker Hub access token
EC2_HOST	52.66.238.68
EC2_USERNAME	ubuntu
EC2_SSH_KEY	Contents of your EC2 .pem key

So your GitHub Actions pipeline can authenticate with both Docker Hub and EC2.

Step 3 — Open Your Workflow in VS Code

Your workflow should be:

```
.github
├── workflows
│   └── ci.yml
```

Open:

```
.github/workflows/ci.yml
```

For the next step, we'll use the complete CI/CD workflow.

Replace the existing workflow with:

name: React CI/CD

on:

push:

branches:

- main

pull_request:

branches:

- main

jobs:

build-and-deploy:

runs-on: ubuntu-latest

steps:

1. Checkout source code

- name: Checkout Code

uses: actions/checkout@v4

2. Setup Node.js

- name: Setup Node.js

uses: actions/setup-node@v4

with:

node-version: 22

cache: npm

3. Install dependencies

- name: Install Dependencies
- run: npm ci

4. Build React application

- name: Build React Application
- run: npm run build

5. Login to Docker Hub

- name: Login to Docker Hub
- uses: docker/login-action@v3
- with:
 - username: \${{ secrets.DOCKERHUB_USERNAME }}
 - password: \${{ secrets.DOCKERHUB_TOKEN }}

6. Build Docker image

- name: Build Docker Image
- run: |
 - docker build -t \${{ secrets.DOCKERHUB_USERNAME }}/reactapp:latest .

7. Push Docker image to Docker Hub

- name: Push Docker Image
- run: |
 - docker push \${{ secrets.DOCKERHUB_USERNAME }}/reactapp:latest

8. Deploy to EC2

- name: Deploy to EC2
- uses: appleboy/ssh-action@v1.2.2
- with:
 - host: \${{ secrets.EC2_HOST }}
 - username: \${{ secrets.EC2_USERNAME }}
 - key: \${{ secrets.EC2_SSH_KEY }}
 - script: |
 - docker pull \${{ secrets.DOCKERHUB_USERNAME }}/reactapp:latest
 - docker stop reactapp || true
 - docker rm reactapp || true
 - docker run -d --name reactapp -p 80:80 \${{ secrets.DOCKERHUB_USERNAME }}

Before pushing, verify these GitHub Secrets

DOCKERHUB_USERNAME

DOCKERHUB_TOKEN
EC2_HOST
EC2_USERNAME
EC2_SSH_KEY

For your new EC2:

EC2_HOST = 52.66.238.68
EC2_USERNAME = ubuntu

Then save the file and run:

```
git add .  
git commit -m "Add CI CD deployment"  
git push origin main
```

After the push, go to:

GitHub → Actions → React CI/CD

You should see the pipeline execute:

```
Checkout  
↓  
Node.js Setup  
↓  
npm ci  
↓  
React Build  
↓  
Docker Login  
↓  
Docker Build  
↓  
Docker Push  
↓  
EC2 Deployment
```

If **all steps are green**, open:

<http://52.66.238.68>

and your React application should be live from the newly deployed container.

Next, let's **test the complete CI/CD pipeline** end-to-end.

Step 1 — Make a small change in your React project

Open your React project in VS Code and change something visible on the page.

For example, change a heading to:

```
<h1>DevOps CI/CD Deployment</h1>
```

You can make any simple visible change.

Step 2 — Save the file

Save your React file:

Ctrl + S

Step 3 — Commit and push

Open the VS Code terminal:

```
git status
```

Then:

```
git add .  
git commit -m "Test automated deployment"
```

Then:

```
git push origin main
```

Step 4 — Watch GitHub Actions

Go to:

GitHub → Your Repository → Actions

You should see:

React CI/CD

Click the running workflow.

You should see:

Checkout Code	✓
Setup Node.js	✓
Install Dependencies	✓
Build React Application	✓
Login to Docker Hub	✓
Build Docker Image	✓
Push Docker Image	✓
Deploy to EC2	✓

The most important step is:

Deploy to EC2 ✓

Step 5 — Verify the EC2 container

SSH into EC2:

```
ssh -i "C:\Users\DELL\Downloads\react-deployment.pem" ubuntu@52.66.238.68
```

Then:

```
docker ps
```

You should see:

```
reactapp
```

You can also check:

```
docker images
```

You should see:

```
rakshith006/reactapp latest
```

Step 6 — Verify the updated application

Open:

```
http://52.66.238.68
```

Your **new React change** should be visible.

If this works, you've demonstrated the complete pipeline:

